

Universidad Complutense de Madrid

Modelos de Redes Neuronales

Máster en Metodología de las Ciencias del Comportamiento y de la Salud

Práctica 2. TRANSFERENCIA DE APRENDIZAJE Y EVALUACIÓN DE MODELOS

Valeria González Baz

Julio 2025

Índice general

1. Evaluar la red neuronal VGG16 entrenada con Imagenet para diagnosticar glaucoma **2**

1.1. Determinar el rendimiento de la red VGG16 -con los pesos resultantes de su entrenamiento con Imagenet- para diagnosticar el glaucoma (si/no) en las retinografías de la base de datos ACRIMA 2

2. Transferencia de aprendizaje **6**

2.1. Utilizar transferencia de aprendizaje para crear la nueva red VGG16-glaucoma congelando todas las capas convolucionales de VGG16 y entrenando la nueva red con las imágenes de ACRIMA; obtener y aportar descripción gráfica de la red resultante, curvas de aprendizaje y un índice de su rendimiento en ACRIMA. 6

2.2. Determinar el rendimiento de VGG16-g obtenida en 2.1 para el diagnóstico de glaucoma con las retinografías de Rim-One-R2 11

2.3. Indicar para 2 imágenes elegidas al azar cuáles son las zonas de cada imagen más determinantes del diagnóstico emitido por la red. 15

3. Estudio comparativo de rendimiento **17**

3.1. Comparar el rendimiento de VGG16-glaucoma en el diagnóstico de glaucoma en ACRIMA y en Rim- One-R2, utilizando el área bajo la curva ROC como indicador del rendimiento 17

3.2. Ídem utilizando otro índice de rendimiento a elegir (justificando la elección) 19

3.3. Comentar la concordancia o discrepancia en las conclusiones obtenidas con ambos Índices de rendimiento. 20

Resumen Ejecutivo

Esta práctica ha tenido como objetivo principal analizar el rendimiento de un modelo de red neuronal convolucional, concretamente VGG16, para el diagnóstico de glaucoma a partir de imágenes de retinografías. Para ello, se ha empleado una estrategia de *transferencia de aprendizaje*, partiendo de los pesos preentrenados en ImageNet y adaptando la red a la tarea específica mediante un reentrenamiento sobre la base de datos **ACRIMA**.

En una primera fase, se congelaron las capas convolucionales de VGG16 y se añadió una cabeza densa personalizada para adaptar la red a la clasificación binaria entre ojos normales y glaucomatosos. El modelo fue entrenado con imágenes de ACRIMA, alcanzando un rendimiento muy elevado, con valores de precisión superiores al 90 %, una sensibilidad del 87 % y un AUC de 0.9710. Estos resultados reflejan que el modelo fue capaz de aprender representaciones visuales relevantes para la detección del glaucoma en el conjunto de entrenamiento.

Sin embargo, la segunda parte de la práctica planteaba un reto de generalización, este era el de aplicar el modelo entrenado sobre ACRIMA a una nueva base de datos externa, **RIM-ONE-r2**, sin ningún reentrenamiento adicional. En este contexto, los resultados fueron drásticamente distintos: el *accuracy* cayó al 48.35 %, el *F1-score* descendió a 0.2900 y el *AUC* se situó en tan solo 0.5467, apenas por encima del azar. La matriz de confusión reveló una sensibilidad extremadamente baja (19 %), indicando que el modelo tendía a ignorar numerosos casos reales de glaucoma.

Para completar el análisis, se aplicó la técnica de *Grad-CAM* a varias imágenes de prueba, con el fin de visualizar las zonas que activaban la red. Aunque en algunos casos las activaciones se concentraban alrededor del nervio óptico (lo esperable clínicamente), también se observaron mapas ruidosos o poco interpretables, especialmente en el conjunto externo, lo que sugiere una falta de consistencia en las representaciones aprendidas.

Finalmente, se compararon dos métricas de rendimiento: el AUC y el F1-score. Ambas coincidieron en su diagnóstico: un rendimiento excelente en el conjunto de entrenamiento y una caída severa en el conjunto externo. Esta concordancia refuerza la conclusión general de la práctica, **aunque la transferencia de aprendizaje puede ser muy eficaz dentro del mismo dominio, los modelos deben validarse en conjuntos de datos variados y realistas para garantizar su utilidad clínica**.

En resumen, esta práctica ha puesto de manifiesto tanto el potencial como los límites del aprendizaje profundo en contextos médicos. Si bien es posible alcanzar altos niveles de rendimiento en entornos controlados, la falta de generalización entre bases de datos evidencia la necesidad de desarrollar modelos más robustos, diversos y clínicamente validados.

1 Evaluar la red neuronal VGG16 entrenada con Imagenet para diagnosticar glaucoma

1.1. Determinar el rendimiento de la red VGG16 -con los pesos resultantes de su entrenamiento con Imagenet- para diagnosticar el glaucoma (si/no) en las retinografías de la base de datos ACRIMA

En este primer apartado se evalúa el rendimiento de la arquitectura **VGG16**, una red convolucional profunda ampliamente utilizada, preentrenada sobre el conjunto de datos *ImageNet*. La finalidad es determinar en qué medida dicha red, sin ningún tipo de ajuste ni reentrenamiento, es capaz de abordar el problema de clasificación binaria (glaucoma / no glaucoma) sobre la base de datos **ACRIMA**, compuesta por retinografías etiquetadas con diagnóstico clínico.

ImageNet es un conjunto masivo y heterogéneo, con más de 14 millones de imágenes distribuidas en 1.000 categorías, en el que VGG16 fue entrenada originalmente para tareas de clasificación general. La red aprendió representaciones jerárquicas de características visuales útiles en distintos contextos, pero *no especializadas* en el análisis de imágenes médicas. En este experimento, se conserva la estructura y los pesos originales de VGG16, y se añade una única capa densa de salida con activación sigmoide, inicializada de forma aleatoria. Esta configuración se utiliza como un extractor de características visuales fijo.

La evaluación se realiza directamente sobre el conjunto de test de ACRIMA, sin ajustar los pesos de ninguna capa. El bloque de código correspondiente es el siguiente:

```
loss, acc = model.evaluate(test_generator, verbose=1)
print(f"[INFO] Precisión del modelo (accuracy): {acc * 100:.2f}%")
print(f"[RESULTADO] Accuracy: {acc:.4f}")
print(f"[RESULTADO] Loss: {loss:.4f}")
```

Listing 1.1: Evaluación directa del modelo VGG16 sin reentrenamiento

Los resultados obtenidos indican una **precisión del 55.32 %** y una **función de pérdida (loss) de 1.7433**. Estos valores, próximos al azar, reflejan que el modelo no está extrayendo características relevantes del dominio clínico para realizar una clasificación fiable. El bajo rendimiento sugiere una falta de capacidad discriminativa ante el cambio de dominio, dado que las representaciones aprendidas

con ImageNet no están alineadas con las particularidades visuales de las retinografías.

La **matriz de confusión** apoya esta conclusión: el modelo muestra una notable confusión entre clases, clasificando erróneamente 28 casos positivos (glaucoma) como negativos (normales), y 35 casos negativos como positivos. Esta simetría en los errores es consistente con un comportamiento cercano al azar.

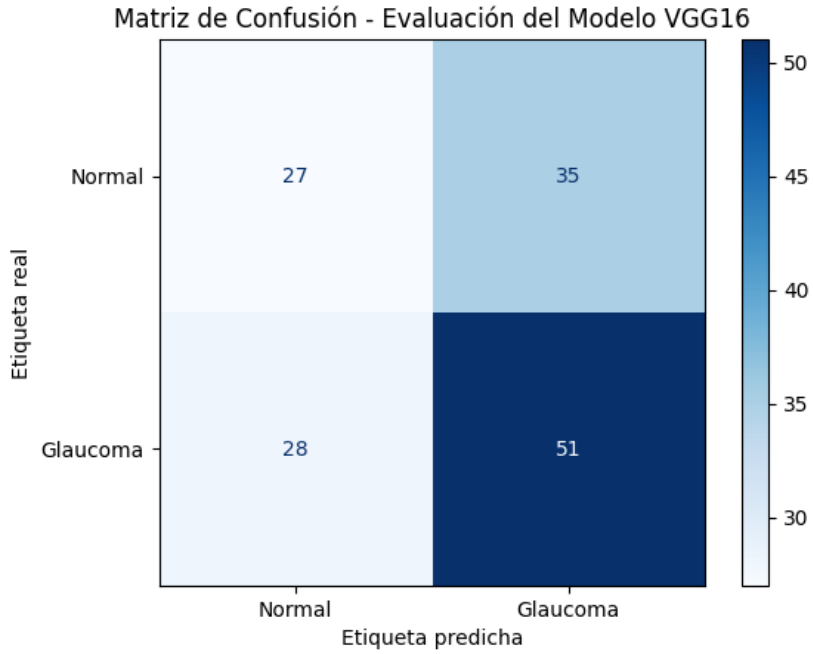


Figura 1.1: Matriz de Confusión. Evaluación directa de ACRIMA con Imagenet

Además, la **curva ROC** obtenida evidencia una falta de separabilidad entre clases. El área bajo la curva (AUC) es de **0.582**, un valor apenas superior al umbral teórico del 0.5 que representaría una clasificación puramente aleatoria. La forma de la curva es próxima a la diagonal, lo que confirma la ausencia de capacidad predictiva significativa.

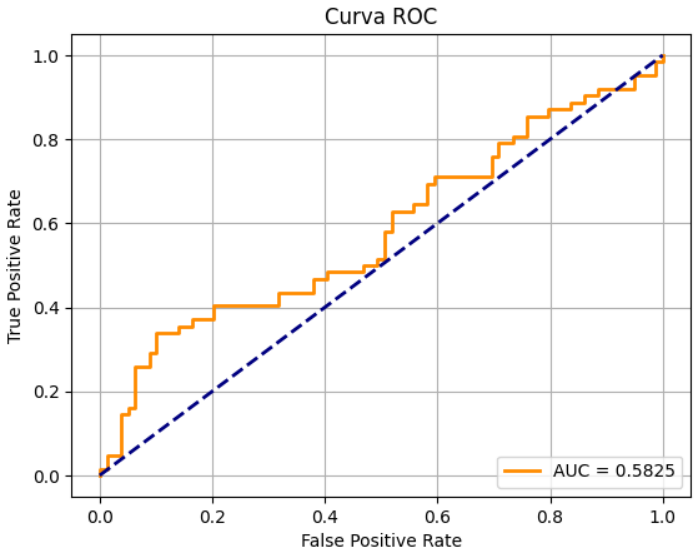


Figura 1.2: Curva Roc. Evaluación directa ACRIMA desde Imagenet

En conjunto, estos resultados demuestran que, pese a su profundidad y eficacia en tareas generales, la red VGG16 preentrenada no es capaz de generalizar eficazmente al dominio oftalmológico sin un proceso de adaptación. Esta limitación puede atribuirse tanto a la discrepancia entre los dominios de origen y destino, como a la inicialización aleatoria de la capa de salida, cuyos pesos no han sido optimizados, además, se observan bastantes errores en la clasificación:

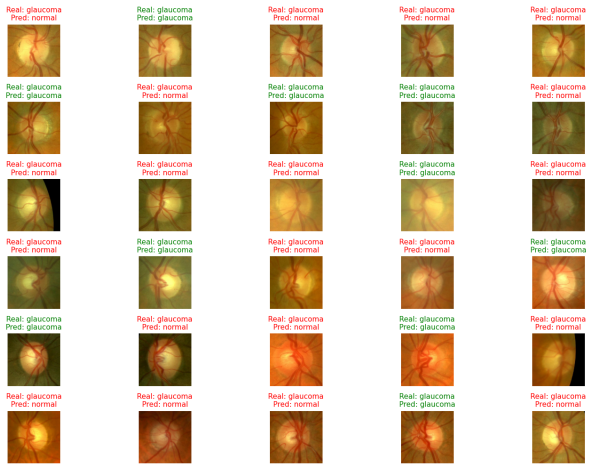


Figura 1.3: Ejemplos de clasificación ACRIMA con pesos de Imagenet

Este análisis resalta la necesidad de aplicar una estrategia de **transferencia de aprendizaje**, que permita ajustar parcialmente el modelo a las características específicas del nuevo problema. Esta estrategia será abordada en el siguiente apartado mediante el congelamiento de capas convolucionales

y el entrenamiento de una nueva cabeza de clasificación sobre ACRIMA.

En definitiva, este primer experimento cumple una función metodológica fundamental: sirve como línea base para evaluar el impacto de la transferencia de aprendizaje en los apartados posteriores. Demuestra que, aunque VGG16 es una arquitectura potente, no puede aplicarse directamente a un dominio clínico sin una adaptación adecuada. Esta observación es coherente con la literatura sobre aprendizaje profundo, el rendimiento de un modelo preentrenado depende en gran medida de la similitud entre el dominio fuente (ImageNet) y el dominio destino (retinografías médicas). Por tanto, esta fase inicial no debe interpretarse como un fallo del modelo, sino como un paso necesario para justificar el enfoque de reentrenamiento parcial que se desarrollará a continuación.

2 Transferencia de aprendizaje

2.1. Utilizar transferencia de aprendizaje para crear la nueva red VGG16-glaucoma congelando todas las capas convolucionales de VGG16 y entrenando la nueva red con las imágenes de ACRIMA; obtener y aportar descripción gráfica de la red resultante, curvas de aprendizaje y un índice de su rendimiento en ACRIMA.

Tras comprobar en el apartado anterior que la red VGG16 preentrenada sobre ImageNet no es capaz de generalizar directamente al dominio clínico del diagnóstico de glaucoma, se procede ahora a aplicar una estrategia de **transferencia de aprendizaje supervisada**. Este enfoque consiste en reutilizar los conocimientos generales ya aprendidos por la red (estructuras visuales de bajo nivel) y adaptarlos a una nueva tarea específica mediante un reentrenamiento parcial.

En particular, se parte nuevamente de la arquitectura **VGG16**, cargada sin su parte superior (*top*), manteniendo los pesos preentrenados sobre ImageNet. Todas las capas convolucionales del modelo se **congelan**, lo que implica que sus pesos permanecerán fijos durante el entrenamiento. Sobre esta base se construye una nueva **cabeza densa personalizada**, que actúa como clasificador binario. Esta cabeza está compuesta por capas densas con activaciones `ReLU`, normalización por lotes (`BatchNormalization`) y capas de `Dropout` para evitar sobreajuste, culminando en una salida de una sola neurona con activación sigmoide.

Este nuevo modelo, denominado **VGG16-g**, se entrena exclusivamente sobre las imágenes de la base **ACRIMA (partición de entrenamiento)**, utilizando técnicas estándar de aumento de datos y validación cruzada. Se emplea un **callback de ModelCheckpoint** para guardar automáticamente la mejor versión del modelo en función de la métrica de validación. El conjunto de test se reserva para evaluar el rendimiento final del modelo una vez entrenado.

Esta aproximación permite mantener la potencia de las representaciones visuales generales aprendidas por VGG16, mientras se adapta su salida a la tarea específica de diagnóstico de glaucoma. Se espera, por tanto, una mejora significativa en métricas como la precisión, sensibilidad y área bajo la curva ROC (AUC), respecto al modelo evaluado en el apartado 1.1.

Durante el proceso de entrenamiento, se registraron las curvas de precisión por época para los subconjuntos de entrenamiento y validación, así como la precisión por lote (*batch*) dentro de cada época. La Figura 2.1 muestra la evolución de estas tres curvas.

Las precisiones por época presentan una tendencia claramente ascendente, estabilizándose a partir de la época 20 en torno al 90 %. No se aprecia una divergencia significativa entre entrenamiento y validación, lo que sugiere que el modelo mantiene una buena capacidad de generalización y no evidencia sobreajuste. La curva por batch, aunque más ruidosa como es habitual, sigue en general la misma trayectoria, lo que refuerza la estabilidad del proceso de optimización.

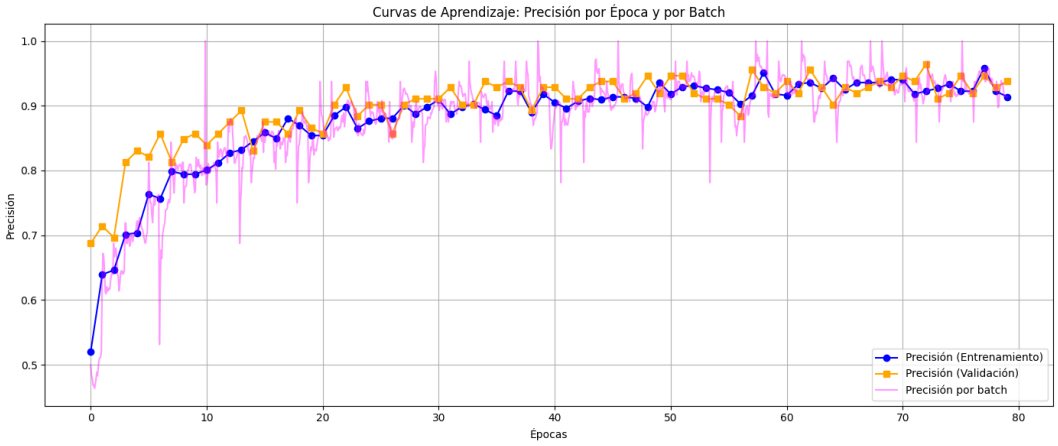


Figura 2.1: Curvas de precisión por época (entrenamiento y validación) y precisión por batch.

Para evaluar la existencia de sobreajuste durante el entrenamiento del modelo, se aplicó una prueba estadística basada en ventanas móviles de tamaño 5 (`window_size = 5`). En cada ventana, se comparó la precisión media del conjunto de entrenamiento y del conjunto de validación mediante una **prueba t de Welch para muestras independientes**.

Se consideró que hay evidencia de sobreajuste cuando se cumplen simultáneamente las dos condiciones siguientes:

- La precisión media en entrenamiento supera a la de validación por al menos 0.02 (`difference_threshold = 0.02`).
- La diferencia es estadísticamente significativa (`p-value <0.05`).

En la Figura 2.2, se muestran las curvas de precisión para entrenamiento y validación, y puede observarse que, si bien existen diferencias entre ambas curvas, estas no cumplen ambos criterios durante ningún periodo prolongado. Como apoyo a esta interpretación, se muestra en la Tabla 2.1 una selección de los valores p obtenidos por época en el análisis estadístico.

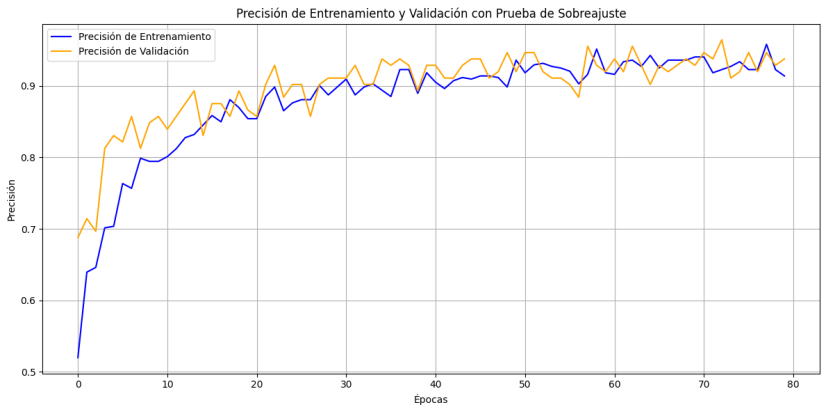


Figura 2.2: Detección estadística de sobreajuste mediante análisis por ventana móvil.

Cuadro 2.1: Resultados de la prueba t por ventana móvil: valores p y diferencias de precisión (train - val)

Época	**p-value**	**diff**	**Época**	**p-value**	**diff**
5	0.0465	-0.1062	18	0.3599	-0.0130
6	0.0515	-0.0843	19	0.6741	-0.0055
7	0.0360	-0.0894	20	0.2214	-0.0108
8	0.0083	-0.0821	21	0.3863	-0.0081
9	0.0105	-0.0707	22	0.5903	-0.0064
10	0.0020	-0.0579	23	0.3042	-0.0172
11	0.0017	-0.0539	24	0.3300	-0.0163
12	0.0043	-0.0429	25	0.2197	-0.0190
13	0.0004	-0.0496	26	0.0379	-0.0226
14	0.0027	-0.0510	27	0.3065	-0.0146
15	0.0369	-0.0355	28	0.4305	-0.0088
16	0.0485	-0.0311	29	0.3921	-0.0097
17	0.0607	-0.0272	76	0.3231	-0.0109

Aunque algunas épocas muestran diferencias estadísticamente significativas (por ejemplo, épocas 7–13), la diferencia de precisión es negativa o menor al umbral mínimo establecido, por lo que **no se detectó sobreajuste estadísticamente significativo** según el criterio adoptado.

Una vez completado el entrenamiento, el mejor modelo (seleccionado mediante validación cruzada) se evaluó sobre el conjunto independiente de test. La matriz de confusión obtenida se muestra en la Figura 2.3. El modelo clasificó correctamente 128 de las 141 imágenes, lo que supone una precisión global (*accuracy*) del **90.78 %**. Además, la precisión o valor predictivo positivo (**PPV**) fue del **96 %**, la **sensibilidad (TPR)** del **87 %** y la **especificidad (TNR)** también del **87 %**. Estas métricas reflejan

una buena capacidad del modelo tanto para detectar correctamente los casos glaucomatosos como para evitar falsos positivos en sujetos normales. El rendimiento general es bueno, aunque sigue siendo necesario tener en cuenta el posible impacto a nivel clínico de estos errores residuales.

```
Matriz de Confusión:
Normal clasificado como normal (TN): 69
Normal clasificado como glaucoma (FP): 10
Glaucoma clasificado como normal (FN): 3
Glaucoma clasificado como glaucoma (TP): 59

Accuracy: 0.91
Precisión (PPV): 0.96
Sensibilidad (TPR): 0.87
Especificidad (TNR): 0.87
```

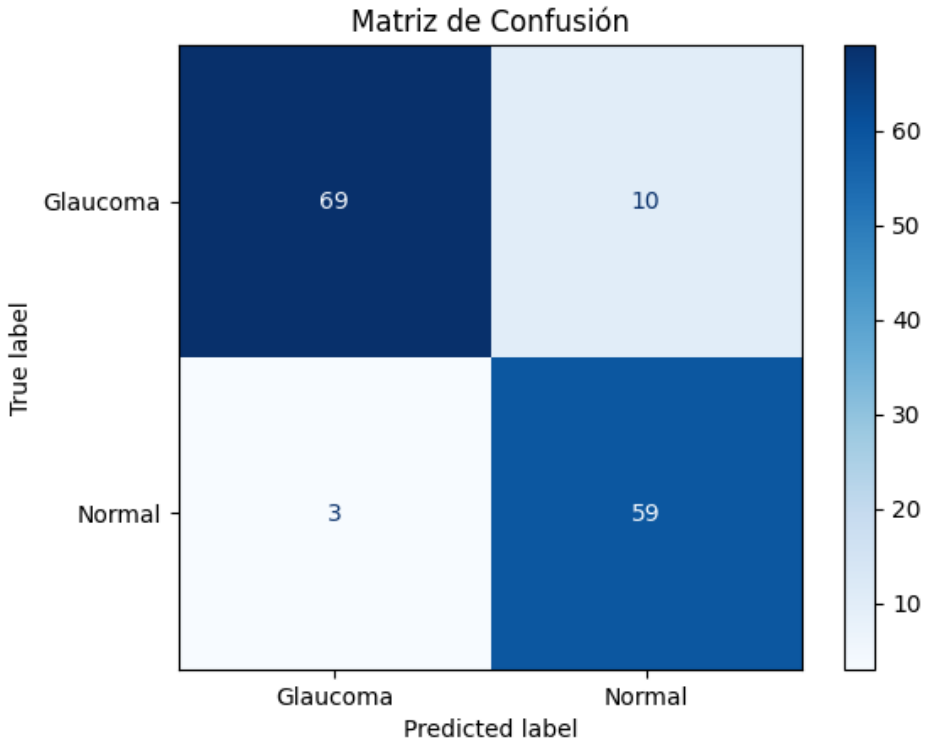


Figura 2.3: Matriz de confusión sobre el conjunto de test (ACRIMA).

En esta evaluación, el área bajo la curva ROC (AUC) alcanzó un valor de **0.9710**, lo que evidencia una capacidad discriminativa muy elevada del modelo. Este indicador sugiere que, en la gran mayoría de los casos, el sistema asigna una probabilidad más alta a imágenes glaucomatosas que a imágenes normales, cumpliendo así eficazmente su función de clasificación.

Además, se calculó el **umbral de decisión óptimo** utilizando el **índice de Youden**, cuyo valor máximo fue de **0.8906**. Este se alcanzó en el umbral **0.9315**, punto en el que se logra un equilibrio óptimo entre **sensibilidad** (90.32 %) y **especificidad** (98.73 %). Este punto representa la mayor separación entre la tasa de verdaderos positivos y la de falsos positivos, siendo especialmente útil en escenarios clínicos donde la prioridad puede variar: ya sea minimizar los falsos negativos para no omitir casos reales de glaucoma, o reducir los falsos positivos para evitar diagnósticos erróneos en pacientes sanos.

```
Índice de Youden máximo: 0.8906
Umbral óptimo (threshold): 0.9315
Sensibilidad óptima: 0.9032
Especificidad óptima: 0.9873
```

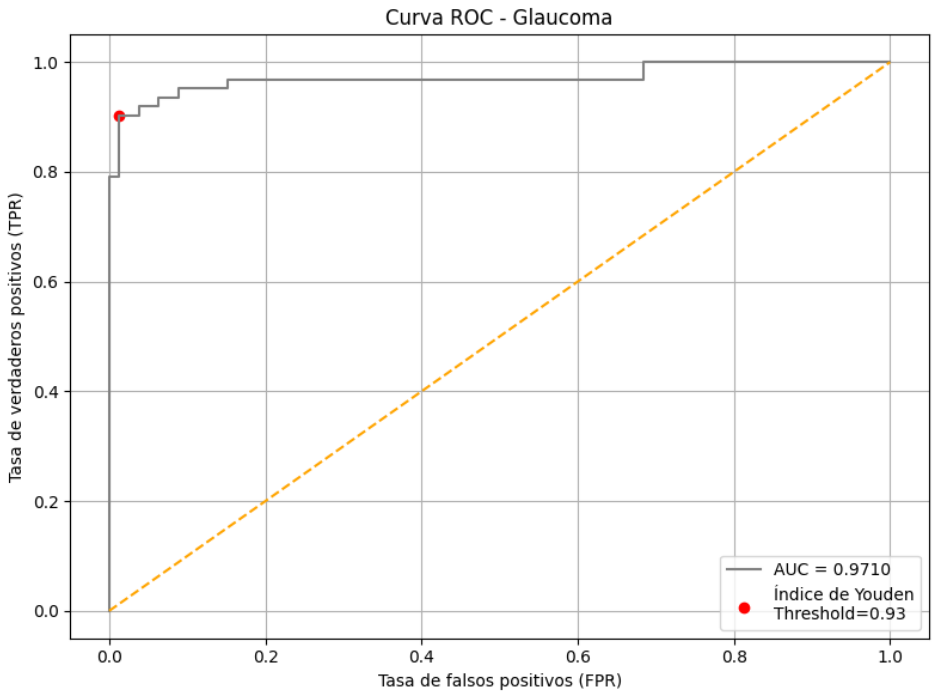


Figura 2.4: Curva ROC con índice de Youden sobre el conjunto de test.

Por último, en la Figura 2.5, se muestran ejemplos de imágenes clasificadas por el modelo sobre una muestra de 30 casos del conjunto de test. El modelo clasificó correctamente 24 de ellas y cometió errores en 6, distribuidos entre ambas clases. Este análisis cualitativo permite visualizar los aciertos y errores del sistema y refuerza la validez de su rendimiento en condiciones reales.

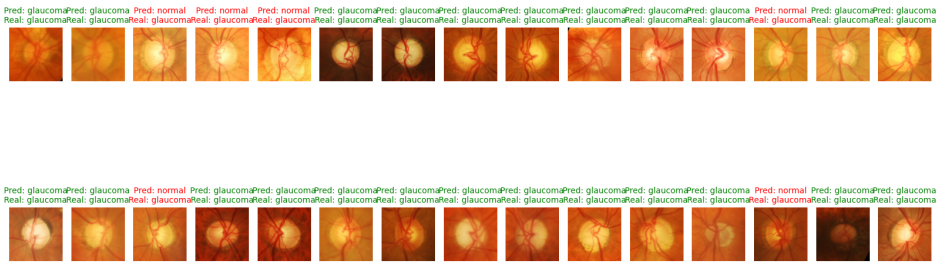


Figura 2.5: Ejemplos visuales de predicciones: clasificaciones correctas (verde) y errores (rojo).

2.2. Determinar el rendimiento de VGG16-g obtenida en 2.1 para el diagnóstico de glaucoma con las retinografías de Rim-One-R2

Tras el entrenamiento del modelo VGG16-g utilizando las imágenes de la base de datos ACRIMA, se plantea en este apartado evaluar su rendimiento sobre un conjunto diferente: la base de datos *RIM-ONE-R2*, que también contiene retinografías destinadas al diagnóstico del glaucoma. El objetivo de esta evaluación es comprobar si el modelo, una vez ajustado a un conjunto concreto, es capaz de mantener un rendimiento aceptable cuando se enfrenta a imágenes procedentes de una fuente distinta, aunque perteneciente al mismo dominio clínico.

Este análisis resulta especialmente relevante en el contexto de aplicaciones médicas, donde es común encontrar variaciones entre bases de datos debidas a diferencias en los dispositivos de captura, condiciones de iluminación, resolución o criterios de anotación. Como se verá a continuación, el modelo no logra trasladar adecuadamente su rendimiento al nuevo conjunto, lo que pone de manifiesto ciertas limitaciones en su capacidad de generalización.

Para llevar a cabo esta evaluación, se reutiliza directamente el modelo previamente entrenado sobre la base de datos ACRIMA. Concretamente, se carga el mejor modelo guardado durante el proceso de entrenamiento, el cual fue almacenado automáticamente mediante el uso de `ModelCheckpoint`. El modelo se aplica ahora, sin ningún tipo de ajuste adicional, sobre el generador de imágenes construido a partir del directorio correspondiente a la base *RIM-ONE-R2*. A continuación, se muestra el fragmento de código utilizado para este fin:

```
# 1. Cargar el modelo del entrenamiento con ACRIMA
from tensorflow.keras.models import load_model

model_acrima = load_model(os.path.expanduser("~/Desktop/best_model_realista.h5"))

```

```
# 2. Evaluar el modelo con el nuevo dataset
loss, acc = model_acrima.evaluate(rimone_generator, verbose=1)
print(f"[RESULTADO] Accuracy: {acc:.4f}")
print(f"[RESULTADO] Loss: {loss:.4f}")
```

Este procedimiento permite obtener de forma directa el valor de `accuracy` (precisión) y `loss` (función de pérdida) al aplicar el modelo sobre la nueva base. Estos indicadores proporcionan una primera medida cuantitativa del rendimiento del modelo en un entorno distinto al de entrenamiento original. Como se discutirá en los apartados siguientes, los valores obtenidos revelan una caída notable en la capacidad de clasificación del modelo, lo que sugiere una generalización limitada.

Matriz de confusión y métricas de clasificación

Para evaluar de forma más detallada el comportamiento del modelo sobre la base de datos *RIM-ONE-R2*, se ha generado la matriz de confusión correspondiente, la cual permite observar el número de aciertos y errores cometidos por la red en función de la clase real y la clase predicha.

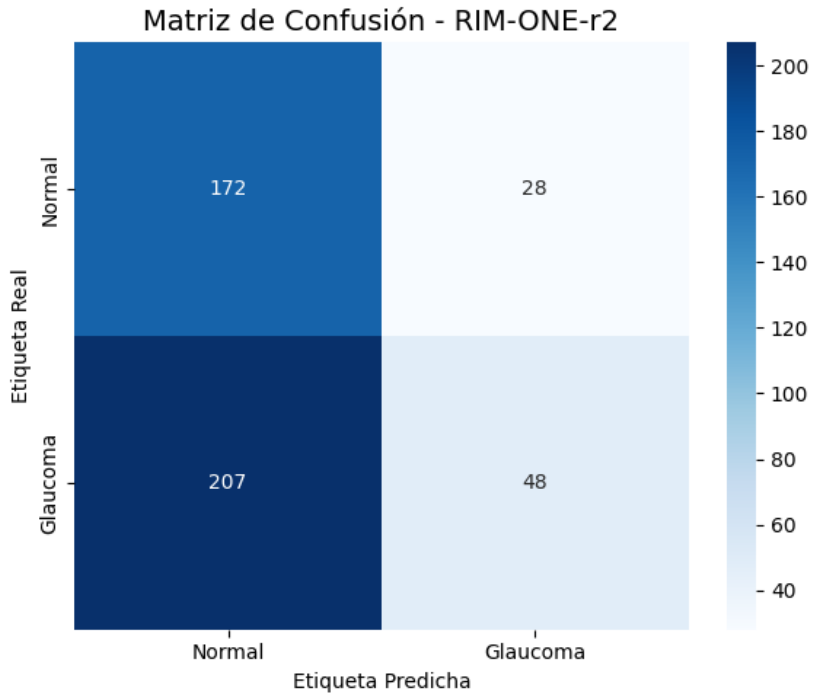


Figura 2.6: Matriz de Confusión

Los resultados obtenidos son los siguientes:

- **True Negatives (TN)**: 172 imágenes normales correctamente clasificadas como normales.

- **False Positives (FP)**: 28 imágenes normales clasificadas erróneamente como glaucomatosas.
- **False Negatives (FN)**: 207 imágenes de glaucoma clasificadas como normales.
- **True Positives (TP)**: 48 imágenes de glaucoma correctamente identificadas.

A partir de estos valores, se derivan las siguientes métricas:

- **Accuracy (precisión global)**: 0.48
- **Precisión (PPV)**: 0.63
- **Sensibilidad (TPR / Recall)**: 0.19
- **Especificidad (TNR)**: 0.86

Tal y como reflejan estos resultados, el modelo presenta una alta especificidad (capacidad para detectar correctamente los casos normales), pero una sensibilidad extremadamente baja. Esto significa que el modelo tiende a clasificar la mayoría de los casos como "normales", pasando por alto una gran cantidad de verdaderos positivos (glaucomas reales), lo cual resulta especialmente problemático en un contexto clínico, donde los falsos negativos pueden tener consecuencias graves.

Curva ROC y área bajo la curva (AUC)

Para completar el análisis cuantitativo del rendimiento del modelo sobre la base RIM-ONE-r2, se ha calculado la curva ROC (Receiver Operating Characteristic), que muestra la relación entre la tasa de verdaderos positivos (sensibilidad) y la tasa de falsos positivos (1 - especificidad) para diferentes valores del umbral de decisión.

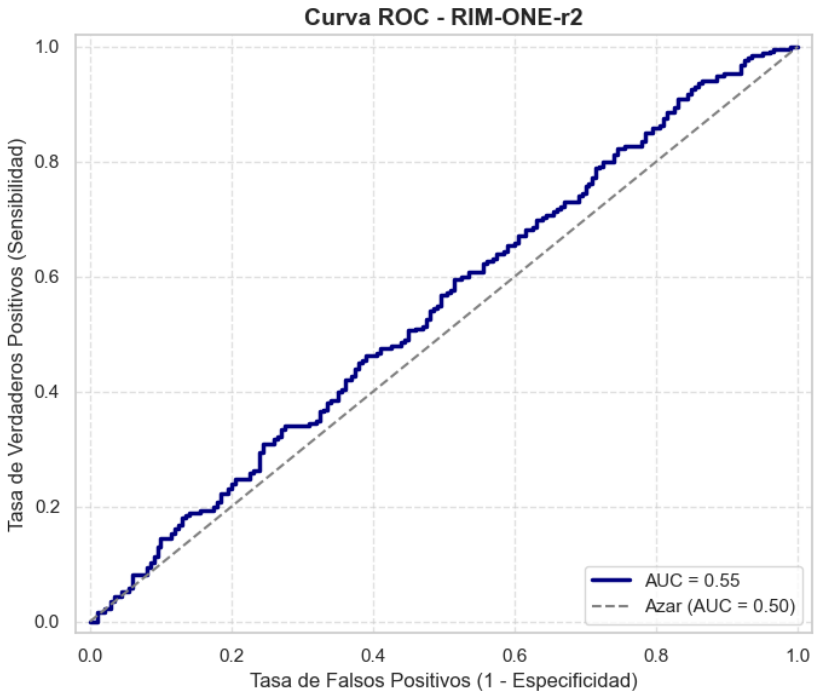


Figura 2.7: Curva ROC

En la gráfica anterior se representa la curva ROC obtenida para el conjunto RIM-ONE-r2, junto con la línea diagonal gris que representa el rendimiento del azar (AUC = 0.50). El área bajo la curva (AUC) obtenida ha sido de **0.5467**, lo que indica que el modelo apenas supera el rendimiento esperado por una clasificación aleatoria.

Además, el umbral de decisión óptimo calculado según el índice de Youden es de **0.04**, lo que sugiere que para maximizar la sensibilidad sin sacrificar completamente la especificidad, el modelo debe operar con un umbral muy bajo. Esta necesidad de ajustar el umbral tan lejos del valor convencional (0.5) es un indicio de desbalance o sesgo en las predicciones generadas por el modelo.

En conjunto, tanto el valor del AUC como la forma de la curva ROC confirman que el modelo no es capaz de discriminar eficazmente entre retinografías normales y glaucomatosas cuando se enfrenta a un conjunto externo como RIM-ONE-r2. Esto refuerza la conclusión previa acerca de la falta de generalización del modelo entrenado exclusivamente con datos de ACRIMA.

Ejemplo de clasificación de retinografías

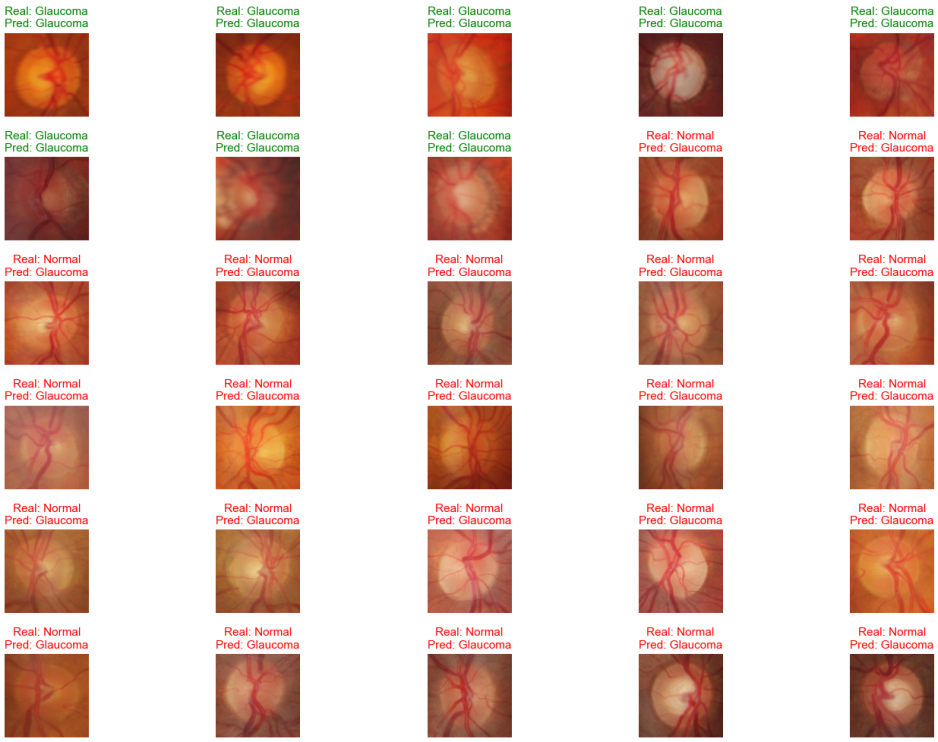


Figura 2.8: Clasificación

2.3. Indicar para 2 imágenes elegidas al azar cuáles son las zonas de cada imagen más determinantes del diagnóstico emitido por la red.

Con el objetivo de interpretar visualmente el funcionamiento interno del modelo y comprender mejor las regiones de la imagen que influyen en sus decisiones, se ha aplicado la técnica *Grad-CAM* (Gradient-weighted Class Activation Mapping).

Esta técnica permite generar un mapa de calor que resalta las zonas de la imagen que más han contribuido al diagnóstico emitido por la red neuronal. En la Figura siguiente se presentan los resultados obtenidos al aplicar Grad-CAM sobre 4 imágenes seleccionadas al azar (2 correspondientes a la clase *Normal* y 2 a la clase *Glaucoma*), mostrando en cada caso la imagen original y la correspondiente superposición del mapa de activación.

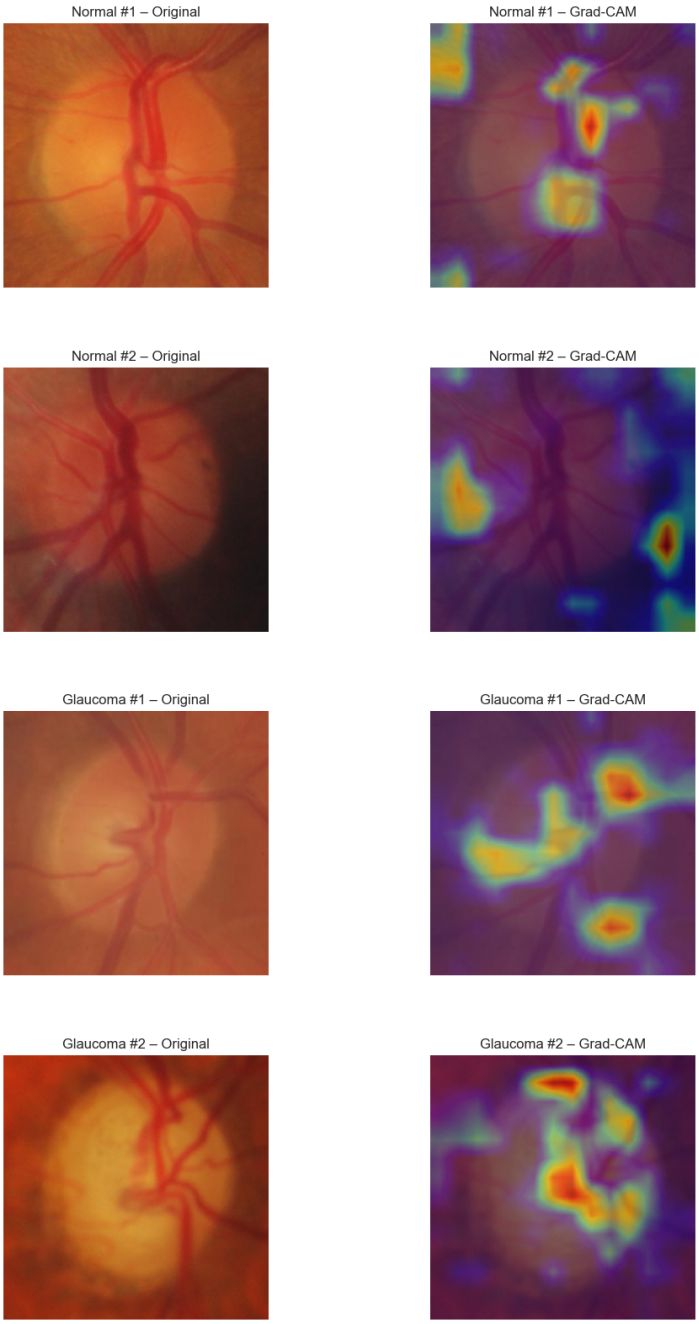


Figura 2.9: Grad - CAM

En las imágenes clasificadas como **Normales**, las activaciones tienden a distribuirse de forma más difusa, sin un patrón especialmente marcado. En cambio, en las imágenes etiquetadas como **Glaucoma**, se observan activaciones más localizadas en regiones cercanas al nervio óptico, lo cual es coherente con la zona de interés clínica para el diagnóstico de esta patología.

3 Estudio comparativo de rendimiento

3.1. Comparar el rendimiento de VGG16-glaucoma en el diagnóstico de glaucoma en ACRIMA y en Rim- One-R2, utilizando el área bajo la curva ROC como indicador del rendimiento

Para analizar la capacidad de generalización del modelo VGG16-g más allá del conjunto de entrenamiento, se ha comparado el rendimiento en términos del área bajo la curva ROC (AUC) entre la base de datos original (ACRIMA) y la base externa (RIM-ONE-r2). Esta métrica es especialmente útil en contextos de clasificación binaria, ya que mide la capacidad del modelo para distinguir entre ambas clases sin depender de un umbral específico.

La comparación se ha realizado utilizando el siguiente código:

```
print(f"AUC en ACRIMA: {roc_auc:.4f}")
print(f"AUC en RIM-ONE-r2: {auc_rimone:.4f}")
print(f"Diferencia absoluta de AUCs: {abs(roc_auc - auc_rimone):.4f}")
```

Y ha generado la siguiente salida:

```
AUC en ACRIMA: 0.9710
AUC en RIM-ONE-r2: 0.5467
Diferencia absoluta de AUCs: 0.4243
```

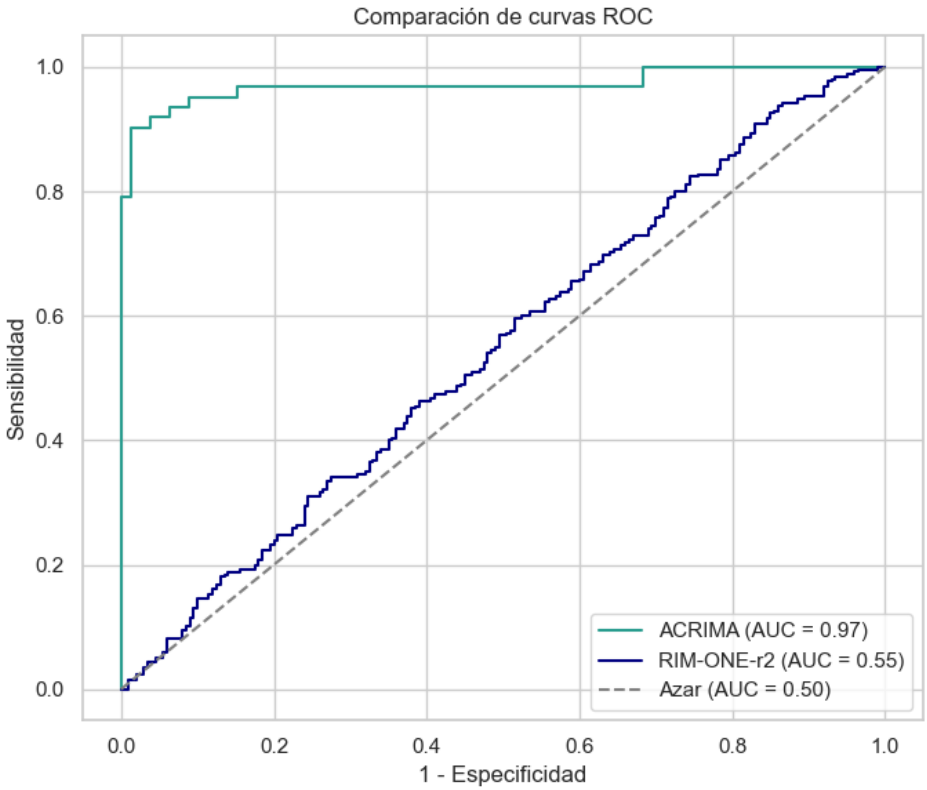


Figura 3.1: Comparativa entre curvas ROC

En la Figura anterior se representan las curvas ROC obtenidas para ambas bases de datos. Los resultados muestran una diferencia significativa de rendimiento: mientras que en la base de datos ACRIMA el modelo alcanza un AUC de 0.9710 —indicador de una excelente capacidad discriminativa—, en la base RIM-ONE-r2 el AUC desciende hasta 0.5467, apenas por encima del azar (0.50). La diferencia absoluta de 0.4243 pone de manifiesto una pérdida importante en la eficacia del modelo al enfrentarse a un conjunto de datos externo.

Esta caída de rendimiento sugiere que el modelo ha aprendido a detectar patrones específicos de la base ACRIMA, pero no ha logrado extraer representaciones suficientemente generales como para adaptarse adecuadamente a otras colecciones de retinografías. Entre los posibles factores que explican esta discrepancia se encuentran diferencias en la calidad de las imágenes, en las condiciones de adquisición, en el preprocesado o incluso en los criterios de etiquetado entre ambas bases.

En consecuencia, este análisis evidencia la necesidad de validar los modelos clínicos sobre múltiples bases de datos independientes antes de considerarlos aptos para su aplicación real.

3.2. Ídem utilizando otro índice de rendimiento a elegir (justificando la elección)

Además del área bajo la curva ROC (AUC), resulta relevante complementar el análisis del rendimiento del modelo con otra métrica que tenga en cuenta tanto la precisión como la sensibilidad. Para ello, se ha calculado el **F1-score**, definido como la media armónica entre la precisión (PPV) y la sensibilidad (Recall). Esta métrica es especialmente útil en problemas de clasificación binaria con clases desbalanceadas, ya que penaliza los errores en cualquiera de las dos direcciones.

El cálculo del F1-score se ha realizado mediante la función `f1_score` de la librería `scikit-learn`, sobre las etiquetas reales y predicciones binarias del modelo en ambos conjuntos de datos. El código empleado ha sido:

```
from sklearn.metrics import f1_score

# Para ACRIMA
f1_acrima = f1_score(y_true, y_pred)

# Para RIM-ONE-r2
y_pred_rimone = (y_pred_probs_rimone > 0.5).astype(int)
f1_rimone = f1_score(y_true_rimone, y_pred_rimone)

print(f"F1-score en ACRIMA: {f1_acrima:.4f}")
print(f"F1-score en RIM-ONE-r2: {f1_rimone:.4f}")
```

Y ha generado los siguientes resultados:

```
F1-score en ACRIMA: 0.9008
F1-score en RIM-ONE-r2: 0.2900
```

La diferencia de rendimiento entre ambos conjuntos es, una vez más, considerable. El valor de F1-score en ACRIMA (0.9008) confirma el excelente comportamiento del modelo en el conjunto de entrenamiento, donde es capaz de equilibrar adecuadamente precisión y sensibilidad. Sin embargo, en RIM-ONE-r2 el F1-score cae hasta 0.2900, lo que indica un fuerte deterioro en su capacidad de detección eficaz cuando se enfrenta a datos externos.

A diferencia del AUC, que mide la capacidad discriminativa en general, el F1-score evalúa directamente el equilibrio entre los aciertos y errores en un punto de decisión concreto (umbral 0.5). Esto permite reforzar la conclusión de que el modelo, pese a tener un rendimiento excelente sobre ACRIMA, no logra mantener su comportamiento al generalizar a nuevos entornos. La discrepancia entre ambas bases, ya apuntada en el análisis del AUC, vuelve a confirmarse desde esta perspectiva alternativa.

3.3. Comentar la concordancia o discrepancia en las conclusiones obtenidas con ambos Índices de rendimiento.

En los apartados anteriores se ha evaluado el rendimiento del modelo VGG16-g utilizando dos métricas complementarias: el área bajo la curva ROC (AUC) y el F1-score. Ambas proporcionan información valiosa, aunque desde perspectivas diferentes: el AUC evalúa la capacidad general del modelo para distinguir entre clases sin depender de un umbral de decisión, mientras que el F1-score resume el equilibrio entre precisión y sensibilidad en un punto de decisión específico.

Los resultados obtenidos muestran una clara concordancia entre ambas métricas:

- En la base de datos **ACRIMA**, el modelo alcanza un AUC de **0.9710** y un F1-score de **0.9008**, lo que refleja un rendimiento excelente y consistente.
- En la base de datos **RIM-ONE-r2**, ambos indicadores caen de forma notable: el AUC se reduce a **0.5467** y el F1-score desciende a **0.2900**, lo que revela una pérdida clara de capacidad discriminativa y de equilibrio entre aciertos y errores.

Esta coincidencia en las conclusiones refuerza la validez del diagnóstico: el modelo, aunque eficaz en el conjunto con el que fue entrenado, no logra generalizar su comportamiento a un entorno externo. La fuerte caída en ambos indicadores deja claro que el rendimiento observado en ACRIMA no es representativo de un comportamiento robusto frente a datos clínicos nuevos, lo cual limita su aplicabilidad real.

En definitiva, la lectura conjunta del AUC y del F1-score ofrece una visión coherente y complementaria del modelo, esto es, no solo disminuye su capacidad de distinguir entre clases, sino también su fiabilidad al emitir predicciones en entornos distintos al original. Esto pone de manifiesto la importancia de validar modelos de aprendizaje automático sobre múltiples bases de datos independientes antes de considerar su uso clínico o su despliegue práctico.

Anexo: Código fuente de las redes utilizadas

Código de la red VGG16 preentrenada con ImageNet (apartado 1.1)

```
import os
import numpy as np
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
```

```

# === MODELO VGG16 PREENTRENADO ===

# 1. Cargar VGG16 sin la parte superior , con pesos de ImageNet
base_model = VGG16( include_top =False , weights='imagenet', input_shape =(224 , 224, 3))

# 2. Congelar TODAS las capas
for layer in base_model.layers:
    layer.trainable = False

# 3. Añadir una cabeza de clasificación mínimamente imprescindible
x = base_model .output
x = Flatten ()(x)
output = Dense (1, activation ='sigmoid')(x) # Se aplana la salida y se conecta a una única neurona con activación sigmoide para la clasificación binaria
model = Model(inputs= base_model .input , outputs=output)

# 4. Compilar (pero SIN entrenar)
model.compile(optimizer=Adam(1e-4), loss='binary_crossentropy', metrics =['accuracy'])

# 5. Obtener un resumen del modelo
model.summary ()
```

Código de la red VGG16-g con transferencia de aprendizaje (apartado 2.1)

```
# 1. Cargamos VGG16 como extractor de características
base_model = VGG16(include_top=False, weights='imagenet', input_shape=(224, 224, 3))

# 2. Congelamos las capas convolucionales
for layer in base_model.layers:
    layer.trainable = False

# 3. Añadimos una nueva cabeza densa personalizada a la red
#    si aumentamos el tamaño de la cabeza puede dar overfitting

x = GlobalAveragePooling2D()(base_model.output)
x = BatchNormalization()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x) # si reducimos el dropout puede dar overfitting
x = Dense(64, activation='relu')(x)
x = Dropout(0.2)(x)
output = Dense(1, activation='sigmoid')(x) # salida binaria

```

```
# 4. Creamos el modelo final
model = Model(inputs=base_model.input, outputs=output)

# 5. Compilamos el modelo
model.compile(optimizer=Adam(1e-4), loss='binary_crossentropy', metrics=['accuracy
    '])

# 6. Resumen del modelo
model.summary()

# 7. Callbacks para guardar el mejor modelo y registrar el entrenamiento
checkpoint = ModelCheckpoint(
    filepath=os.path.expanduser("~/Desktop/best_model_realista.h5"),
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)

# 8. Logger para registrar el entrenamiento en CSV
csv_logger = CSVLogger(os.path.expanduser("~/Desktop/training_log_realista.csv"))
```